Why Agentic Design is Necessary for Data Analytics: The Limits of Multi-task Standalone LLM Inference

Anonymous Author
Anonymous Institution

Abstract

Agentic Large Language Models (LLMs) have gained attention for their ability to handle complex tasks through tool use, reasoning, and adaptive decision-making. However, the need for agentic design, beyond standard non-agentic approaches, has not been clearly established. In this work, we provide theoretical support for why agentic LLMs are necessary in data analytics. We demonstrate that, without agentic or reasoning-based mechanisms, using LLMs in standalone inference mode, even after pre-training and post-training enhancements such as supervised fine-tuning (SFT), or reinforcement learning from human feedback (RLHF), cannot reliably perform tasks in multi-task data science settings. Moreover, we theoretically and empirically show that mitigating this shortcoming would require modifications in the pre-training stage, which diverges from the standard LLM training paradigm. Our results characterize these limitations of standalone LLM inference and clarify where these approaches fall short in multi-task data science applications.

1 INTRODUCTION

LLMs have recently been extended from text generators to more agentic systems. Agentic LLMs can reason through problems, plan intermediate steps, and call external tools to complete tasks. Frameworks such as multi-component prompting (MCP) make it possible for an LLM to behave more like an intelligent decision-maker, enabling flexible interaction with data and computation. These advances have sparked growing interest in applying agentic LLMs to data science and analytics, where workflows often require multi-step reasoning, integration of structured and unstructured data, and specialized computational tools. In this setting, the appeal of agentic design lies in its ability to combine reasoning, tool use, and adaptive workflows to perform complex analyses more effectively than standard, non-agentic approaches.

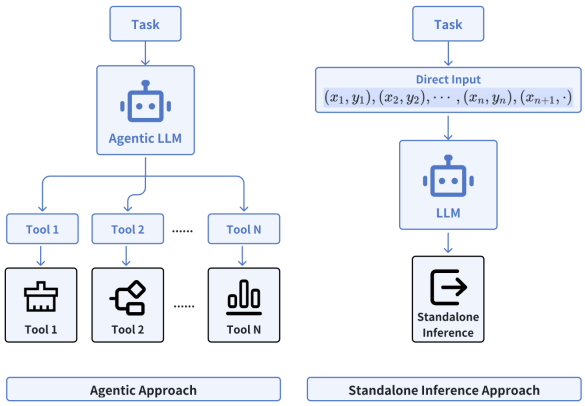

Figure 1: Agentic Approach vs Standalone Inference Approach

While agentic design appears to be a natural solution, its necessity in statistics and data science has not been clearly established. In particular, it remains unclear whether the standard pre-training and post-training pipeline of LLMs can adequately support such tasks. In this paper, we examine the limitations of the standalone inference pipeline (i.e., non-reasoning and non-agentic) in data science settings, and we use these results to justify the need for agentic design.

For clarity, in this work we use **agentic** to denote approaches that invoke reasoning or tool-calling during inference, and **standalone inference** (or **standalone LLM**) to denote the direct use of an LLM in inference mode after pre-training and post-training, without enabling reasoning or tool use. See Figure 1 for the difference.

Since the data science domain is highly quantitative, it provides a natural setting for rigorously analyzing the inherent capacity of standalone LLMs to meet the

Preliminary work. Under review by AISTATS 2026. Do not distribute.

Confidential reviewer copy. Under review by AISTATS 2026. DO NOT SHARE.

demands of such tasks. Prior work has highlighted several promising capabilities, most notably in the setting of in-context learning (ICL) (Brown et al., 2020). Once pretrained on a large corpus, LLMs can often solve newly encountered tasks from only a few training examples, without requiring updates to their parameters (Xie et al., 2021; Akyürek et al., 2022; Von Oswald et al., 2023; Zhang et al., 2023a). One of the most important findings is that LLMs can learn fairly complex functions purely through pre-training and exposure to training examples in-context at inference time, making them promising for data analytics tasks.

More specifically, it is well known that for classical learning problems, say linear regression, pre-trained LLMs know how to learn the best linear regression algorithms in-context (Garg et al., 2022b; Akyürek et al., 2022; Bai et al., 2024). That is, given a dataset $\{(x_i, y_i)\}_{i=1}^n$ as the prompt where $y_i = \omega^\top x_i + \epsilon_i$, ω and $x_i \in \mathbb{R}^d$, and the noise $\{\epsilon_i\}_{i=1}^n$ are independently and identically sampled from a Gaussian distribution, an LLM, denoted by $f_\theta(\cdot)$, pre-trained on regression datasets, can take as input the prompt $\{(x_i, y_i)\}_{i=1}^n$ as context together with a new feature x_{n+1}, and output

$$\hat{y}_{n+1} = f_\theta(x_1, y_1, \cdots, x_n, y_n, x_{n+1}).$$

It has been shown that the LLM's output $\hat{y}_{n+1}$ performs on par with the Bayes optimal estimator in terms of minimizing the mean squared error (Garg et al., 2022b) in the single task setting. Moreover, Transformer-based LLMs are known to be expressive enough to represent off-the-shelf learning algorithms (Bai et al., 2024; Lin et al., 2023), including ordinary least squares, ridge regression, and neural networks. There has also been prior work on how standalone LLMs handle multi-task settings (Panwar et al., 2023; He et al., 2025) but these studies typically consider scenarios where only the task parameters are sampled from a distribution. In contrast, our setting involves sampling both the task types and their parameters from a distribution. We refer the readers to Appendix A for related work.

Contribution. Our contributions are threefold.

- **Agentic Design is Necessary**. By constructing a standard analytical multi-task environment, we provide theoretical and empirical evidence that non-reasoning approaches such as ICL, SFT, and RLHF under the standard pipeline are insufficient in multi-task settings. This provides a clear justification for adopting an agentic design.
- **Task Representation Affects Standalone Inference**. We further examine the multi-task learning setting for standalone inference, and identify a key factor influencing multi-task downstream performance. We theoretically show that when task embeddings from the penultimate layer of the Transformer are not well-separated, downstream performance is degraded.
- **Alternative Standalone Inference Pipeline**. Meanwhile, to improve the standalone inference's capacity, we propose an alternative pre-training pipeline that incorporates an auxiliary loss during pre-training, enabling the LLM to learn better task representations and improving the downstream performance. Empirical and theoretical evidence are provided to support the pipeline.

2 FORMULATIONS: LLM FOR DATA SCIENCE

In this section we introduce the setup for applying LLMs for data-science tasks. Imagine a user submits a request like "help me predict next month's sales," along with the relevant dataset. There are two ways the model might respond. An agentic LLM will examine the data, interpret the user's goals, and devise a step-by-step plan: cleaning and preparing the data, engineering features, selecting or fitting a suitable model, and then produce a final forecast. A standalone inference pipeline, by contrast, operates without external tools or planning: it simply feeds the dataset into the pre-trained model and returns a prediction according to its built-in inference routine.

Standalone Inference Pipeline. Given a prompt of the form $P = [x_1, y_1, x_2, y_2, \cdots, x_n, y_n, x_{n+1}]$, where the first part $[x_1, y_1, x_2, y_2, \cdots, x_n, y_n]$ can be interpreted as examples of covariate-label pairs $\{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$, where $y_i = f(x_i)$ for some function f; the LLM aims to use the context $\{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$ to infer the underlying relation between x and y. Given the new covariate x_{n+1} (the last element of the prompt P) the model is expected to produce an output $f_\theta(P) \approx y_{n+1} = f(x_{n+1})$. In other words, the model seeks to learn the task from the prompt and generalize based on the knowledge acquired during pre-training.

Multi-task Environment. In practice, data scientists are often confronted with a variety of tasks, making the multi-task setting a crucial aspect in applying LLMs to data science. We draw a task function f from a mixture distribution $\mathcal{F}$ that represents different categories of tasks, and then generate the prompt $P = [x_1, f(x_1), \ldots, x_n, f(x_n), x_{n+1}]$. To evaluate performance, we sample a prompt–response pair (P, y) from another distribution $\mathcal{I}$, where y is the true re-

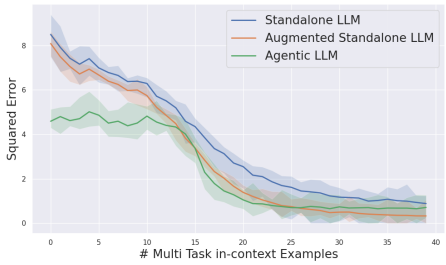

Figure 2: Performance comparison: Agentic vs Standalone LLMs

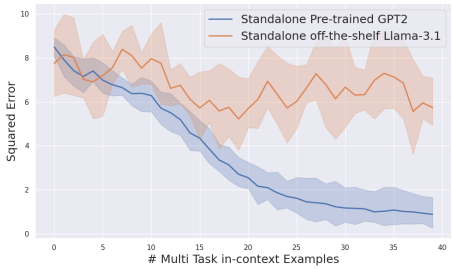

Figure 3: Performance of Standalone Inference for Different LLMs

sponse to P. Note that $\mathcal{I}$'s prompts need not follow the same distribution as those generated from $\mathcal{F}$. We then measure the predictive performance from the expected loss

$$\mathcal{L}_{\mathcal{I}}(\theta) = \mathbb{E}_{(P,y)\sim\mathcal{I}}\big[L(f_\theta(P), y)\big], \tag{1}$$

where $L(\cdot)$ is a loss function to be specified.

Agentic vs Standalone LLM in Multi-task Setting. Figure 2 shows that agentic approaches achieve much stronger performance than standalone inference. The details of the agentic pipeline and experimental setup are provided in Appendix C.2.

Although there is not yet a consensus on the precise formulation of the agentic framework, its capacity is widely acknowledged. Formal analyses of such a framework remain infeasible due to the absence of an agreed-upon design. By contrast, standalone inference has a more established theoretical foundation, making it possible to probe its limitations. In the following sections, we therefore focus on analyzing the limitations of standalone inference in our setting.

Off-the-shelf LLM vs. Self-pretrained LLM.

For standalone inference, we compare Llama 3.1 (Lambert et al., 2024), an off-the-shelf LLM, with GPT-2 (Radford et al., 2018), a smaller-scale model widely used in research for pre-training experiments and reproducible studies, which we train from scratch on our curated datasets. As shown in Figure 3, Llama 3.1 performs significantly worse than our self-pretrained model. Henceforce, to better characterize the limitations of standalone LLMs, we focus on the smaller-scale model, which also allows us to explore a wider range of configurations and conduct more comprehensive comparisons. The implementation details can be found in Appendix C.3.

3 Limitation of Standalone LLMs

3.1 Architecture and Limitation of Standalone LLMs

The LLM's architecture is mathematically complex. To simplify our discussion, we provide an abstraction of the architecture in Figure 4 and explore the properties and limitations of standalone LLM. Given a prompt $P = [x_1, y_1, \cdots, x_{n+1}]$, a Transformer-based LLM can be viewed as a composition of two functions $f_\theta(P) = \xi_\theta \circ \gamma_\theta(P)$. $\gamma_\theta(\cdot)$ corresponds to the first function block, and this block consists of stacked layers, each beginning with masked multi-head self-attention, followed by addition and layer normalization operations, and ending with a feed-forward network; The function $\xi_\theta(\cdot)$ represents a linear function followed by a Softmax or ReLU function, depending on the format of y_n. With such a representation, all prompts of the form $P = [x_1, y_1, \cdots, x_{n+1}]$ will be mapped by $\gamma_\theta(\cdot)$ to a Euclidean space $\mathbb{R}^d$.

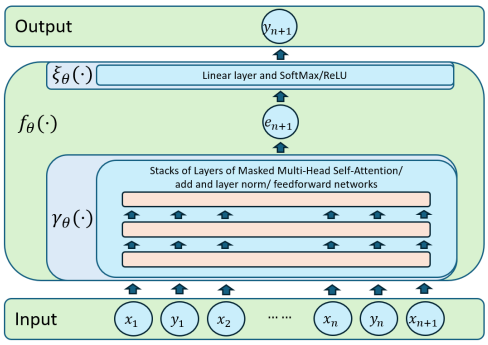

Figure 4: Simplified architecture of Transformer-based ICL

Limitations of the Architecture. The above architecture reveals a limitation of standalone LLMs: although it appears theoretically flexible, as it can take prompts $P = [x_1, y_1, \cdots, x_{n+1}]$ with any number of examples and any relationship between y_i and x_i, so that $f_\theta(P)$ approximates y_n, this flexibility is largely constrained by $\gamma_\theta(\cdot)$. In the decomposition $f_\theta(P) = \xi_\theta \circ \gamma_\theta(P)$, all prompts P are first mapped by $\gamma_\theta(\cdot)$ to the same embedding space $\mathbb{R}^d$, with the final output $\xi_\theta(\gamma_\theta(P))$ produced by the relatively simple function $\xi_\theta(\cdot)$.

In other words, the effectiveness of standalone LLM hinges on $\gamma_\theta(\cdot)$'s ability to distinguish between different tasks of the prompts, and map them to distinct regions in $\mathbb{R}^d$ (as shown in Figure 5). Such limitation

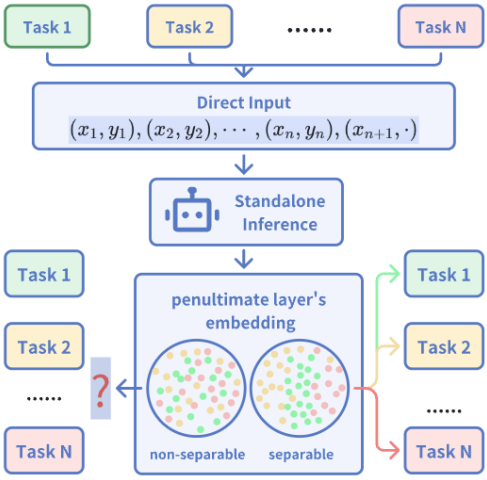

Figure 5: Distribution of Embeddings Affects Task Identification

would not constrain the standalone LLM's capability in single-task learning, but would matter in a multi-task learning setting, as we will see in the next section.

3.2 Standard Setups

Pre-training. In the pre-training stage of LLMs, the pre-training dataset $\mathcal{T}$ is of the form $\mathcal{T} = \{(P_i, y_i)\}_{i=1}^N$, where $(P_i, y_i) = ([x_1^{(i)}, f^{(i)}(x_1^{(i)}), \cdots, x_n^{(i)}, f^{(i)}(x_n^{(i)}), x_{n+1}^{(i)}], f^{(i)}(x_{n+1}^{(i)}))$ for some $n > 0$ and $f^{(i)}(\cdot)$ sampled from the distribution $\mathcal{F}$. Pre-training is done via minimizing the empirical version of the standard pre-training loss

$$\mathcal{L}_{\mathcal{T}}(\theta) = \mathbb{E}_{(P,y)\sim\mathcal{T}}\left[L(f_\theta(P), y)\right], \tag{2}$$

where L is the loss function to be specified. However, in the pre-training stage, the distribution of the task will affect the performance during inference time.

Multi-task Setup. Following Section 2, we consider a multi-task setting that involves three different types of tasks: classification, linear regression, and quadratic regression. This setup aims to better mimic the real-world behavior of LLMs during training and inference, where the pre-training data and the tasks encountered during inference can come from different distributions. With this setting, the prompt is of the form $P = [x_1, f(x_1), \cdots, x_n, f(x_n), x_{n+1}]$, where f is sampled from a mixture distribution of $\{\mathcal{F}_i\}_{i=1}^3$, and $\mathcal{F}_1$, $\mathcal{F}_2$, and $\mathcal{F}_3$ stands for the sets of functions for linear regression, quadratic regression, and classification.

In the pre-training stage, users generate their own data and know the task associated with each prompt. Therefore, the pre-training dataset $\mathcal{T}$ can also have the form $\{(P_i, y_i, z_i)\}_{i=1}^N$, where $z_i = j$ if $f^{(i)}$ is sampled from $\mathcal{F}_j$. We note that although the standard pre-training loss does not involve the category label z, we will later show that the absence of z leads to undesirable inference behavior in our setting.

Environments	LR-S	QR-S	CF-S
Three Tasks	33.36%	36.72%	28.79%
Two Tasks	44.57%	56.82%	N/A
Two Tasks	50.33%	N/A	50.90%

Table 1: Classification accuracy based on the representation $\gamma_\theta(P)$. LR, QR, and CF represent linear regression, quadratic regression, and classification tasks, respectively, with -S indicating results of standalone LLM with the standard pre-training loss. In the three-task model, random guessing results in 33.3% accuracy per task, while in the two-task model (excluding the task marked N/A), random guessing yields 50% accuracy per task.

3.3 Limitations in Task Classification

In this section, we present empirical results and formal theorems demonstrating the limitations of standalone inference in multi-task learning. Specifically, we show that accurate prediction using a standalone LLM hinges on the model's ability to identify the task type from the input P (as shown in Figure 5), because predicting $f(x_{n+1})$ reliably requires the model to determine the correct task type. Under the standard pre-training pipeline, however, LLMs cannot distinguish between task types effectively, and therefore cannot achieve near-optimal performance on such tasks.

From Figure 4, LLMs perform two-stage evaluation $f_\theta(P) = \xi_\theta \circ \gamma_\theta(P)$: all prompts P are first mapped by a complex function $\gamma_\theta(\cdot)$, which is within the function class of multi-head attention networks, to the same embedding space $\mathbb{R}^d$, with the final output $\xi_\theta(\gamma_\theta(P))$ produced by the relatively simple function $\xi_\theta(\cdot)$, which is within the function class of simple neural networks.

Now, let us characterize a scenario that is critical for achieving good downstream prediction performance. Consider two prompts P_1 and P_2 drawn from different tasks. Each prompt is mapped by $\gamma_\theta(\cdot)$ into an embedding in $\mathbb{R}^d$. If the distributions of $\gamma_\theta(P_1)$ and $\gamma_\theta(P_2)$ are too similar, then a simple downstream function $\xi_\theta(\cdot)$ will not be able to tell them apart, and thus cannot handle the tasks correctly. For strong downstream predictive performance, $\gamma_\theta(\cdot)$ must inherently produce distinguishable representations for different tasks, such that $\xi_\theta(\cdot)$ can learn P_1 and P_2 differently.

However, from Table 1, we find that in all of our multi-task settings, the resulting embedding generated by $\gamma_\theta(\cdot)$ cannot be separated well. For prompts from different tasks, we take out the embedding of the last token in the penultimate layer after inference, and train a neural network as a classifier to predict the label z. Across different tasks, the best classifier performs only slightly better than random guessing. This partly explains the inferior performance of standalone LLMs

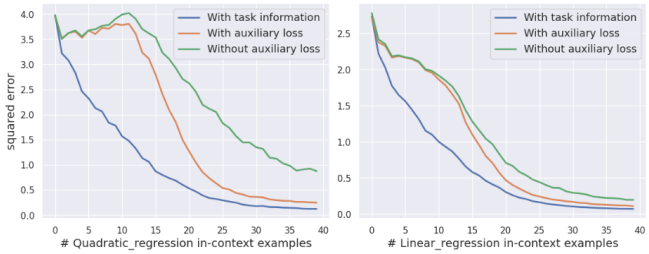

Figure 6: Comparison in the multi-task setting for Standard vs Augmented Pre-training Pipeline. The blue, orange, and green curves represent the LLM's performance when pre-trained on a single task, with the auxiliary loss introduced in (6), and with a standard loss, respectively. The results show that the auxiliary loss improves downstream performance.

in Figure 2 (blue curve) and 6 (green curve). Other experimental insights can be found in Section 5.

3.3.1 Theoretical Development

Next, we present the theory that characterizes this phenomenon. We demonstrate that if $\gamma_\theta(\cdot)$ cannot map the prompts of different tasks to well-separated embeddings, then the downstream predictive performance will be affected. The theory is developed in the two-task setting for clarity of presentation, but the results extend naturally to the general multi-task case. For the prompt-response pair (P, y) with label $z \in \{0, 1\}$, we define the embedding as $\kappa = \gamma_\theta(P)$ and make the following assumptions on κ.

Assumption 3.1. Let $z \sim \text{Bernoulli}(p)$ with $P(z = 0) = p$. Conditioned on z, the random vector $\kappa \in \mathbb{R}^d$ follows one of two distributions: $\kappa \mid z = 0 \sim \mathcal{K}_0$, $\kappa \mid z = 1 \sim \mathcal{K}_1$. Unconditionally, κ follows the mixture distribution $\kappa \sim \mathcal{K}$.

We need another mild assumption before we present the theorem.

Assumption 3.2. For a constant $c > 0$, with probability at least $1 - \delta$, for $\kappa \sim \mathcal{K}$ we have

$$|\mathbb{E}[y|\kappa, z = 0] - \mathbb{E}[y|\kappa, z = 1]| > c \tag{3}$$

Assumption 3.2 suggests that with high probability, different prompt P will correspond to different y $(= f(x_{n+1}))$. Our multi-task setting naturally satisfies this assumption because, for different f, the $f(x)$s will be different most of the time.

Next, we define the hypothesis class $\mathcal{H}$ to be functions consisting of multi-layer neural networks with the last layer being a Softmax layer. We present the next fundamental theorem.

Theorem 3.3. *we have*

$$\begin{aligned}&\mathbb{E}_{(P,y)\sim\mathcal{I}}\left[L(f_\theta(P), y)\right] \geq \textit{irreducible error} \\ &+ \frac{1}{2}c^2 \cdot \inf_{f\in\mathcal{H}} \mathbb{E}_{(\kappa,z)\sim(\mathcal{K},\mathcal{Z})}[l(f(\kappa), z)] - \frac{1}{4}c^2\delta,\end{aligned} \tag{4}$$

Environments	LR-S	QR-S	CF-S
Pre-training	33.36%	36.72%	28.79%
Post-training SFT	50.40%	42.36%	78.37%
Post-training DPO	72.35%	20.70%	82.25%

Table 2: Classification accuracy based on the mapped representation $\gamma_\theta(P)$. Here -S indicates results of standalone LLM inference. Post-training improves the task classification rate in the three-task setting, but not in a consistent manner. The downstream performance can be found in Figure 7.

where irreducible error $= \mathbb{E}_{\kappa\sim\mathcal{K}_0}[Var(y|\kappa, z = 0)] + \mathbb{E}_{\kappa\sim\mathcal{K}_1}[Var(y|\kappa, z = 1)]$, $L(\cdot)$ *is the squared loss, and* $l(\cdot)$ *is the 0-1 classification loss.*

Remark. Theorem 3.3 provides a lower bound for the predictive performance in the multi-task setting. It implies that if the representation of P follows a distribution such that no simple functions in $\mathcal{H}$ can classify the category z, then $\min_{f\in\mathcal{H}} \mathbb{E}_{(\kappa,z)\sim\mathcal{K}\times\mathcal{Z}}[l(f(x), z)]$ and the prediction loss will be large.

Moreover, we can derive a similar upper bound.

Assumption 3.4. For some $C > 0$, we have $|\mathbb{E}[y|\kappa, z = 0] - \mathbb{E}[y|\kappa, z = 1]| < C$ holds for all $\kappa \in \mathbb{R}^d$.

Theorem 3.5. *we have*

$$\begin{aligned}&\min_{g\in\mathcal{G}} \mathbb{E}_{(P,y)\sim\mathcal{I}}\left[L(g(\gamma_\theta(P)), y)\right] \leq \textit{irreducible error} \\ &+ \frac{1}{4}C^2[1 - (1 - 2\inf_{f\in\mathcal{H}} \mathbb{E}_{(\kappa,z)\sim(\mathcal{K},\mathcal{Z})}[l(f(\kappa), z)])^2]\end{aligned} \tag{5}$$

where irreducible error $= \mathbb{E}_{\kappa\sim\mathcal{K}_0}[Var(y|\kappa, z = 0)] + \mathbb{E}_{\kappa\sim\mathcal{K}_1}[Var(y|\kappa, z = 1)]$, *and* $\mathcal{G} = \{g | g : \mathbb{R}^d \to \mathbb{R}$, *neural networks that maps the embedding* κ *to* $\hat{y}\}$.

Remark. Theorem 3.5 provides an upper bound for $\min_g \mathbb{E}_{(P,y)\sim\mathcal{I}}[L(g(\gamma_\theta(P)), y)]$, the lowest achievable loss for the current representation extractor $\gamma_\theta(\cdot)$, which still depends on the classification loss for tasks.

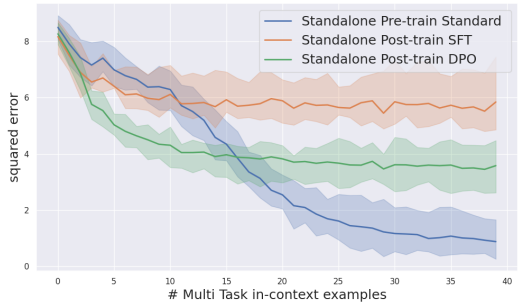

Figure 7: Performance Comparison: Pre-training vs Post-training. Note that we use the standard pre-training pipeline for the blue curve.

3.4 Limitation Persists after Post-training

We identify the same issue after post-training. We employed Supervised Fine-tuning (SFT) and Direct

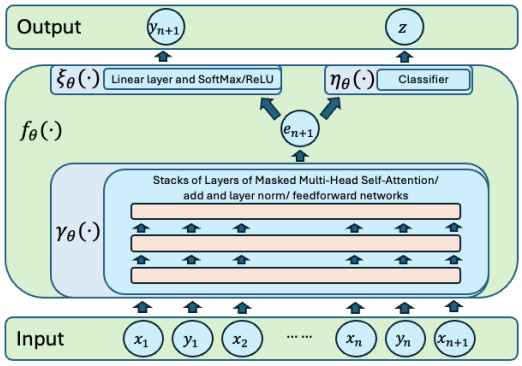

Figure 8: Architecture for the Auxiliary Pre-training Pipeline

Preference Optimization (DPO) for the pre-trained model, and the same issue persists. From Table 2, we observe that post-training yields improved but inconsistent task classification accuracy, and these gains do not translate into better downstream performance. As shown in Figure 7, downstream performance after post-training is worse than that observed at the pre-training stage. Further details of the post-training procedure are provided in Appendix C.4.

4 Resolving the Limitation through Pre-training Objective Design

Next, we provide an alternative pre-training pipeline to improve the performance of the standalone LLM inference. Our proposed approach introduces an auxiliary loss that measures the LLM's ability to classify different tasks during pre-training.

The alternative pipeline comes from the observation that in practice, practitioners often have access to the types of tasks in pre-training. We use this additional information to enhance multi-task learning. From the pre-training loss (2), we instead denote the loss $L(f_\theta(P), y)$ inside the expectation by $L_{main}(f_\theta(P), y)$. Moreover, we impose an auxiliary loss, denoted by $L_{aux}(\cdot,\cdot)$. For a given prompt P, based on the representation $\gamma_\theta(P)$, we can train another classifier $\eta_\theta(P) \in \mathcal{H}$ to predict the corresponding task label z (see the setup in Section 3.2). The auxiliary loss $L_{aux}(\cdot,\cdot)$ is often taken to be the cross-entropy loss, aiming to quantify the LLM's ability to classify tasks. Implied by Theorem 3.3 , we expect that by imposing this auxiliary loss, the LLM can learn a representation $\gamma_\theta(\cdot)$ that better handles downstream tasks.

The new structure is illustrated in Figure 8. To summarize, the alternative pre-training loss can be represented as

$$\mathcal{L}_{\mathcal{T}}(\theta,\phi) = \mathbb{E}_{(P,y,z)\sim\mathcal{T}}[L_{main}(\xi_\theta(\gamma_\theta(P)), y) + \phi \cdot L_{aux}(\eta_\theta(\gamma_\theta(P)), z)], \quad (6)$$

Environments	LR-A	QR-A	CF-A
Three Tasks	93.35%	91.01%	97.54%
Two Tasks	91.71 %	95.05%	N/A
Two Tasks	97.48%	N/A	99.68%

Table 3: Classification accuracy based on the mapped representation $\gamma_\theta(P)$. LR, QR, and CF represent linear regression, quadratic regression, and classification tasks, respectively, with -A indicating results using the auxiliary pre-training loss.

where $\phi \in \mathbb{R}^+$ can be viewed as the hyper-parameter that balances the weight of loss functions.

When evaluating the predictive performance, we use the same loss as in (1), with a different notation

$$\mathcal{L}_{\mathcal{I}}(\theta) = \mathbb{E}_{(P,y)\sim\mathcal{I}}\left[L_{main}(f_\theta(P), y)\right].$$

Empirical Performance. Our alternative pre-training approach achieves improvements over the standard pipeline, consistent with our theoretical findings. As shown in Table 3, the introduction of the auxiliary loss, through training an additional classifier in $\mathcal{H}$ on the representation space, leads to significantly higher accuracy compared with Table 1. This result indicates a much lower classification loss in (4), and consequently, an improved predictive performance: in Figure 6 (orange curve), the introduction of the auxiliary loss is beneficial such that the performance trained with the auxiliary loss outperforms the performance trained with the standard pre-training loss only.

Theoretical Justification. Because of the page limit, we present the theoretical justification of our augmented pipeline in Appendix B.1.

5 Insights from Experiments

In this section, we present additional experiments on standalone LLMs, and provide a clearer picture of their capacities and limitations.

- The first finding is that the relative proportions of pre-training task distributions can negatively impact downstream predictive performance.
- The second finding is that the standard pre-trained LLMs tend to converge toward an "averaged" solution across tasks, which is inherently suboptimal.
- Finally, we find that for standard pre-trained LLMs, the suboptimal downstream performance does not improve even as model scale increases.

We put other related experimental findings and details in Appendix C.5.

5.1 Task Distribution in Pre-training Affects Performance

We find that the distribution of tasks in the pre-training data can negatively influence downstream performance. This is problematic because, in general multi-task data science settings, there is no guarantee that each task is sufficiently represented in the pre-training corpus.

In our experiment, we denote the pre-training datasets as $\mathcal{T}_1 = \{(P_{1,i}, y_{1,i})\}_{i=1}^{N_1}$ for linear regression, $\mathcal{T}_2 = \{(P_{2,i}, y_{2,i})\}_{i=1}^{N_2}$ for quadratic regression, and $\mathcal{T}_3 = \{(P_{3,i}, y_{3,i})\}_{i=1}^{N_3}$ for classification tasks. During pre-training on $\mathcal{T} = \mathcal{T}_1 \cup \mathcal{T}_2 \cup \mathcal{T}_3$, we vary the proportions of each $\mathcal{T}_i$. At evaluation, we study how these ratios affect downstream predictive performance. In particular, in Figure 9, we adjust the proportion of quadratic regression data (33%, 50%, 80%), with the remainder evenly split between linear regression and classification. The results show that performance on quadratic regression improves with more quadratic data, while performance on linear regression declines.

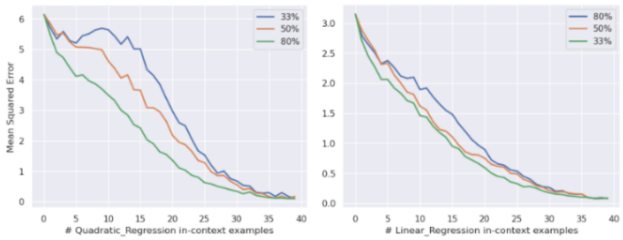

Figure 9: The effect of pre-training distribution on different downstream predictive performance

5.2 The Averaging Behavior

Next, we empirically show that the standard pre-training pipeline leads the LLM to collapse different task distributions into a single one, causing it to learn an averaged function rather than capturing the distinct characteristics of each task.

Multi-task Setup. For better comparison, we take two types of tasks: linear regression, denoted by LR, and quadratic regression, denoted by QR. That is, we consider the prompt-response pair $(P, y) = [(x_1, y_1, \cdots, x_{n+1}], y)$ generated from two types of tasks, for the linear regression we sample $\beta \sim N(0, \tau^2 I_d)$ and $\epsilon_i \sim N(0, \sigma^2 I_d)$, and let $y_i = \beta^\top x_i + \epsilon_i$; For quadratic regression, we have $y_i = \beta_{quad}^\top x_{i,quad} + \epsilon_i = \beta_l^\top x_{i,l} + \beta_n^\top x_{i,n} + \epsilon_i$, where $\beta_{quad}^\top = [\beta_l, \beta_n]^\top$ and $x_{quad}^\top = [x_l, x_n]^\top$, with l and n stand for the linear and non-linear term, respectively. Also notice that the β_{quad} is of the same form of β, but is sampled from a higher-dimensional Gaussian distribution than that of the linear regression. Again for determining the task $f(\cdot)$ s.t. $y_i = f(x_i)$, we sample z from Bernoulli distribution with parameter 0.5, such that $z = 1$ corresponds to QR, and $z = 0$ corresponds to LR. With such information, we can collect the pre-training set $\mathcal{T} = \{P_i, y_i, z_i\}_{i=1}^n$.

Bayesian Optimal Estimator. Given the environment setup and the current prompt P, The Bayesian Optimal Estimator $f^*(P)$ for the prompt $P = [x_1, y_1, \cdots, x_{n+1}]$ is

$$\begin{aligned} f^*(P) &= \mathbb{E}[y_{n+1}|P] \\ &= P(z=0|P) \cdot x_{n+1}^\top \hat{\beta}_{\text{ridge},0} \\ &\quad + P(z=1|P) \cdot x_{n+1,quad}^\top \hat{\beta}_{\text{ridge},1}, \end{aligned}$$

where $\hat{\beta}_{\text{ridge},0} = \left(\boldsymbol{X}^\top \boldsymbol{X} + \frac{\sigma^2}{\tau^2}\boldsymbol{I}\right)^{-1} \boldsymbol{X}^\top \boldsymbol{y}$, $\hat{\beta}_{\text{ridge},1} = \left(\boldsymbol{X}_{quad}^\top \boldsymbol{X}_{quad} + \frac{\sigma^2}{\tau^2}\boldsymbol{I}\right)^{-1} \boldsymbol{X}_{quad}^\top \boldsymbol{y}$, and $\boldsymbol{X}, \boldsymbol{X}_{quad}$, and $\boldsymbol{y}$ are constructed from examples $\{x_i, y_i\}_{i=1}^n$ except x_{n+1} from the prompt P.

Transformer-based LLM vs Bayesian Optimal Estimators. There has been a surging interest in the theoretical community that compares the pre-trained LLM with the Bayesian Optimal Estimator (Xie et al., 2021; Garg et al., 2022b; Panwar et al., 2023). Our multi-task setting can be viewed as a natural setting for this comparison, providing an understanding of when the pre-trained LLM behaves like the Bayesian Optimal Estimators.

More specifically, we define $f_{LR}^*(\cdot)$ and $f_{QR}^*(\cdot)$ to be the Bayesian Optimal Estimator for the linear regression and quadratic regression task, respectively, such that for a given prompt $P = [x_1, y_1, \cdots, x_{n+1}]$,

$$\begin{aligned} f_{LR}^*(P) &= x_{n+1}^\top \hat{\beta}_{\text{ridge},0} \\ f_{QR}^*(P) &= x_{n+1,quad}^\top \hat{\beta}_{\text{ridge},1}. \end{aligned}$$

Furthermore, we define the $f_{\theta,\text{std}}(\cdot)$ and $f_{\theta,\text{aux}}(\cdot)$ to be the LLM pretrained with the standard loss and auxiliary loss. Now, we begin to sample evaluation environments that consist of either only LR tasks or QR tasks. Notice that for the evaluation distribution $\mathcal{I}$, where $(P, y) \sim \mathcal{I}$, it is designed in a way such that the conditional probability $P(z = 0|P)$ is close to 0 or 1, mitigating the possible mixture behavior of two distributions. By such a design, the function

$$\begin{aligned} f^*(P, z) &= \mathbf{1}_{\{z=0\}} x_{n+1}^\top \hat{\beta}_{\text{ridge},0} \\ &\quad + \mathbf{1}_{\{z=1\}} x_{n+1,quad}^\top \hat{\beta}_{\text{ridge},1} \end{aligned}$$

should be close to the Bayesian Optimal Estimator.

Averaging Behavior. With this setup, we can design our experiment and show the average behavior of the LLM under the standard pre-training loss.

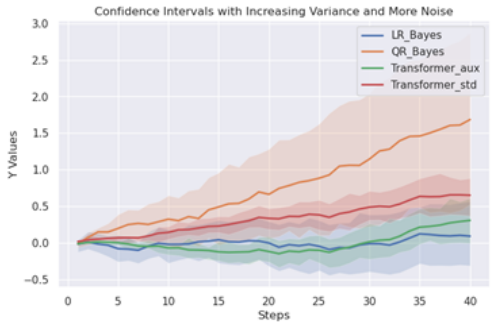

Figure 10: The sampled task is linear regression, so a good estimator should stay close to LR_Bayes. x-axis is the number of in-context samples, and y-axis is the summation of the function's output over previous in-context samples. This graph shows the averaging behavior of the LLM pre-trained with standard loss. In contrast, the LLM pre-trained with the auxiliary loss performs similarly to the Bayes Optimal Estimators. This experiment is averaged over 2000 independent trials.

LLM	**LR Bayes**	**QR Bayes**
$f_{\theta,\mathbf{aux}}(\cdot)$ **under LR**	1.51	36.25
$f_{\theta,\mathbf{aux}}(\cdot)$ **under QR**	12.54	1.73
$f_{\theta,\mathbf{std}}(\cdot)$ **under LR**	19.06	22.75
$f_{\theta,\mathbf{std}}(\cdot)$ **under QR**	19.79	18.86

Table 4: Squared prediction Difference of LLMs to Optimal Estimators. The lower the value, the closer the performance.

We sample prompt $\{P_i\}_{i=1}^n$ from a specific distribution of task, and compare the output of 4 functions $f^*_{LR}(\cdot), f^*_{QR}(\cdot), f_{\theta,\text{aux}}(\cdot)$, and $f_{\theta,\text{std}}(\cdot)$ as the number of in-context sample increases. From Figure 10, we notice that the LLM pre-trained with the auxiliary loss recognizes the task and stays close to the Optimal Bayes Estimator, while the LLM pre-trained with the standard loss acts like the average of two Optimal Bayes Estimators.

Next, we adopt another metric called Squared Prediction Difference (SPD) defined as $\text{SPD}(f, g) = \mathbb{E}_{(P,y)\sim\mathcal{I}}[(f(P)-g(P))^2]$. In the environment that is either linear regression or quadratic regression, we compute the SPD between the LLM and the Bayes Optimal Estimator for LR and QR.

From Table 4, we know that the LLM using the auxiliary loss behaves similarly to the Optimal Bayesian Estimator, while the LLM pre-trained with standard loss deviates from both Optimal Bayesian Estimators. In this multi-task setting, the behavior is different from the ones reported in single-task ICL setting (Akyürek et al., 2022; Garg et al., 2022b; Bai et al., 2024; Panwar et al., 2023).

5.3 Scalability

We next show that the suboptimal performance of the standard pre-trained LLM is not primarily due to model scale. We test three configurations: (1) doubling the number of attention layers, (2) doubling the number of fully connected layers, and (3) doubling both. Each model is trained three times, with 50 independent evaluations per pre-trained model, to reduce randomness from both pre-training and evaluation, and we report the mean and 95% confidence intervals.

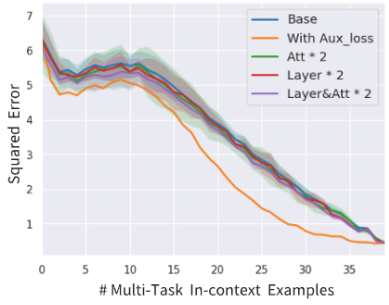

Figure 11: Results on Scalability

Even after scaling the model, the standard pre-training pipeline still underperforms compared to the augmented pipeline. This implies that the standard pre-training pipeline is insufficient for multi-task data science settings. The augmented pre-training pipeline can yield better performance, yet such approaches are not part of standard LLM training.

6 Conclusion

In this work, we examined the capacities and limitations of standalone LLM inference in multi-task data science settings and argued for the necessity of agentic design. We showed both theoretically and empirically that they fail to generalize across heterogeneous tasks. This stems from standard pre-training pipelines, which do not encourage the model to separate task representations and instead yield averaged, suboptimal solutions. The problem persists despite post-training methods such as SFT and RLHF, or scaling the model, underscoring the inadequacy of the standard pipeline and the need for alternative strategies.

To probe this further, we proposed an alternative pre-training pipeline with an auxiliary loss for task identification. This improves task representations and downstream accuracy, but still falls short of even simple agentic approaches, highlighting the importance of developing agentic LLMs for data science tasks.

Overall, our results clarify where standalone inference fails and demonstrate the advantages of agentic approaches that integrate reasoning and tool use. We hope these findings will contribute to further work on principled agentic designs and novel training objectives toward more capable LLMs for data analytics.

References

Achille, A., Lam, M., Tewari, R., Ravichandran, A., Maji, S., Fowlkes, C. C., Soatto, S., and Perona, P. (2019). Task2vec: Task embedding for meta-learning. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 6430–6439.

Ahn, K., Cheng, X., Daneshmand, H., and Sra, S. (2024). Transformers learn to implement preconditioned gradient descent for in-context learning. *Advances in Neural Information Processing Systems*, 36.

Akyürek, E., Schuurmans, D., Andreas, J., Ma, T., and Zhou, D. (2022). What learning algorithm is in-context learning? investigations with linear models. *arXiv preprint arXiv:2211.15661*.

Bai, Y., Chen, F., Wang, H., Xiong, C., and Mei, S. (2024). Transformers as statisticians: Provable in-context learning with in-context algorithm selection. *Advances in neural information processing systems*, 36.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.

Chan, S., Santoro, A., Lampinen, A., Wang, J., Singh, A., Richemond, P., McClelland, J., and Hill, F. (2022). Data distributional properties drive emergent in-context learning in transformers. *Advances in Neural Information Processing Systems*, 35:18878–18891.

Dong, Q., Li, L., Dai, D., Zheng, C., Wu, Z., Chang, B., Sun, X., Xu, J., and Sui, Z. (2022). A survey on in-context learning. *arXiv preprint arXiv:2301.00234*.

Evgeniou, T., Micchelli, C. A., Pontil, M., and Shawe-Taylor, J. (2005). Learning multiple tasks with kernel methods. *Journal of machine learning research*, 6(4).

Garg, S., Tsipras, D., Liang, P., and Valiant, G. (2022a). What can transformers learn in-context? a case study of simple function classes. In *arXiv preprint*.

Garg, S., Tsipras, D., Liang, P. S., and Valiant, G. (2022b). What can transformers learn in-context? a case study of simple function classes. *Advances in Neural Information Processing Systems*, 35:30583–30598.

He, J., Pan, X., Chen, S., and Yang, Z. (2025). In-context linear regression demystified: Training dynamics and mechanistic interpretability of multi-head softmax attention. *arXiv preprint arXiv:2503.12734*.

He, X., Alesiani, F., and Shaker, A. (2019). Efficient and scalable multi-task regression on massive number of tasks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 3763–3770.

Kolar, M. and Liu, H. (2012). Marginal regression for multitask learning. In *Artificial Intelligence and Statistics*, pages 647–655. PMLR.

Lambert, N., Morrison, J., Pyatkin, V., Huang, S., Ivison, H., Brahman, F., Miranda, L. J. V., Liu, A., Dziri, N., Lyu, S., et al. (2024). Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*.

Lin, L., Bai, Y., and Mei, S. (2023). Transformers as decision makers: Provable in-context reinforcement learning via supervised pretraining. *arXiv preprint arXiv:2310.08566*.

Liu, S., Cai, Z., Chen, G., and Li, X. (2024). Towards better understanding of in-context learning ability from in-context uncertainty quantification. *arXiv preprint arXiv:2405.15115*.

Liu, Z., Hu, H., Zhang, S., Guo, H., Ke, S., Liu, B., and Wang, Z. (2023). Reason for future, act for now: A principled architecture for autonomous llm agents. In *Forty-first International Conference on Machine Learning*.

Lu, Z., Pu, H., Wang, F., Hu, Z., and Wang, L. (2017). The expressive power of neural networks: A view from the width. *arXiv preprint arXiv:1709.02540*.

Mahankali, A., Hashimoto, T. B., and Ma, T. (2023). One step of gradient descent is provably the optimal in-context learner with one layer of linear self-attention. *arXiv preprint arXiv:2307.03576*.

Michelucci, U. and Venturini, F. (2019). Multi-task learning for multi-dimensional regression: application to luminescence sensing. *Applied Sciences*, 9(22):4748.

Panwar, M., Ahuja, K., and Goyal, N. (2023). In-context learning through the bayesian prism. In *The Twelfth International Conference on Learning Representations*.

Radford, A., Narasimhan, K., Salimans, T., Sutskever, I., et al. (2018). Improving language understanding by generative pre-training.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I. (2019). Language models are unsupervised multitask learners.

Ruder, S. (2017). An overview of multi-task learning in deep neural networks. *arXiv preprint arXiv:1706.05098*.

Shen, L., Mishra, A., and Khashabi, D. (2023). Do pretrained transformers really learn in-context by gradient descent? *arXiv preprint arXiv:2310.08540*.

Solnon, M., Arlot, S., and Bach, F. (2012). Multi-task regression using minimal penalties. *The Journal of Machine Learning Research*, 13(1):2773–2812.

Von Oswald, J., Niklasson, E., Randazzo, E., Sacramento, J., Mordvintsev, A., Zhmoginov, A., and Vladymyrov, M. (2023). Transformers learn in-context by gradient descent. In *International Conference on Machine Learning*, pages 35151–35174. PMLR.

Wang, H., Pan, Y., Sun, F., Liu, S., Talluri, K., Chen, G., and Li, X. (2024a). Understanding the training and generalization of pretrained transformer for sequential decision making. *arXiv preprint arXiv:2405.14219*.

Wang, S., Yang, C.-H. H., Wu, J., and Zhang, C. (2024b). Bayesian example selection improves in-context learning for speech, text, and visual modalities. *arXiv preprint arXiv:2404.14716*.

Wang, X., Zhu, W., Saxon, M., Steyvers, M., and Wang, W. Y. (2024c). Large language models are latent variable models: Explaining and finding good demonstrations for in-context learning. *Advances in Neural Information Processing Systems*, 36.

Wu, S., Zhang, H. R., and Ré, C. (2020). Understanding and improving information transfer in multi-task learning. *arXiv preprint arXiv:2005.00944*.

Xie, S. M., Raghunathan, A., Liang, P., and Ma, T. (2021). An explanation of in-context learning as implicit bayesian inference. *arXiv preprint arXiv:2111.02080*.

Zhang, R., Frei, S., and Bartlett, P. L. (2023a). Trained transformers learn linear models in-context. *arXiv preprint arXiv:2306.09927*.

Zhang, Y., Zhang, F., Yang, Z., and Wang, Z. (2023b). What and how does in-context learning learn? bayesian model averaging, parameterization, and generalization. *arXiv preprint arXiv:2305.19420*.

Checklist

From the authors: our responses are marked in green.

1. For all models and algorithms presented, check if you include:
   (a) A clear description of the mathematical setting, assumptions, algorithm, and/or model. [Yes/No/Not Applicable]
   (b) An analysis of the properties and complexity (time, space, sample size) of any algorithm. [Yes/No/Not Applicable]
   (c) (Optional) Anonymized source code, with specification of all dependencies, including external libraries. [Yes/No/Not Applicable]

2. For any theoretical claim, check if you include:
   (a) Statements of the full set of assumptions of all theoretical results. [Yes/No/Not Applicable]
   (b) Complete proofs of all theoretical results. [Yes/No/Not Applicable]
   (c) Clear explanations of any assumptions. [Yes/No/Not Applicable]

3. For all figures and tables that present empirical results, check if you include:
   (a) The code, data, and instructions needed to reproduce the main experimental results (either in the supplemental material or as a URL). [Yes/No/Not Applicable]
   (b) All the training details (e.g., data splits, hyperparameters, how they were chosen). [Yes/No/Not Applicable]
   (c) A clear definition of the specific measure or statistics and error bars (e.g., with respect to the random seed after running experiments multiple times). [Yes/No/Not Applicable]
   (d) A description of the computing infrastructure used. (e.g., type of GPUs, internal cluster, or cloud provider). [Yes/No/Not Applicable]

4. If you are using existing assets (e.g., code, data, models) or curating/releasing new assets, check if you include:
   (a) Citations of the creator If your work uses existing assets. [Yes/No/Not Applicable]
   (b) The license information of the assets, if applicable. [Yes/No/Not Applicable]
   (c) New assets either in the supplemental material or as a URL, if applicable. [Yes/No/Not Applicable]

(d) Information about consent from data providers/curators. [Yes/No/Not Applicable]

(e) Discussion of sensible content if applicable, e.g., personally identifiable information or offensive content. [Yes/No/Not Applicable]

5. If you used crowdsourcing or conducted research with human subjects, check if you include:

   (a) The full text of instructions given to participants and screenshots. [Yes/No/Not Applicable]

   (b) Descriptions of potential participant risks, with links to Institutional Review Board (IRB) approvals if applicable. [Yes/No/Not Applicable]

   (c) The estimated hourly wage paid to participants and the total amount spent on participant compensation. [Yes/No/Not Applicable]